PID Tuning in Practice

Contents

1. Overview . . . 2
2. The PID forms: which numbers we tune . . . 3
   - 2.1 The ideal PID controller in parallel form . . . 4
   - 2.2 The standard form: one gain and two times . . . 4
3. Ziegler–Nichols: tuning from one experiment . . . 7
   - 3.1 Why the ideal speed model never oscillates . . . 7
   - 3.2 The measured speed arrives late . . . 8
   - 3.3 Raise the gain until the loop oscillates . . . 9
   - 3.4 The Ziegler–Nichols rules and their price . . . 11
4. Model-based tuning: a bump test and a small model . . . 12
   - 4.1 The bump test . . . 12
   - 4.2 The model and the tuning rule . . . 13
   - 4.3 Three tunings side by side . . . 15
5. Derivative kick and set-point weighting . . . 17
   - 5.1 The heading PID and its test . . . 17
   - 5.2 Derivative kick . . . 17
   - 5.3 Set-point weighting . . . 19
6. Filters inside the loop . . . 21
   - 6.1 The derivative filter . . . 21
   - 6.2 The notch filter . . . 23
7. Summary . . . 27
8. Sources . . . 28
9. Key terms . . . 29

Where this fits:

- **Builds on:** PID control; First-order systems and the time constant; Bode plot; Gain, phase and delay margins.

1. Overview

Key point: A PID controller's three numbers are found by experiments on the robot, either by raising the gain until the loop oscillates (Ziegler–Nichols) or by fitting a small model to one bump test. Practice then changes the textbook PID in three places, so that command steps and sensor noise do not jerk the motors.

The robot of this chapter can follow speed and heading commands. Its PID controller (G-2965) adds three actions on the error: one proportional to it, one to its accumulated area, one to its rate of change. RO-066 added feedforward (G-3070), the force the plan needs sent before any error; anti-windup (G-3078), which stops the integral growing while the drive is at its limit; and cascade control (G-3086), a fast inner loop under a slower outer one. Its speed loop is RO-060's PI controller, with a proportional gain of 20 N per m/s and an integral gain of 10 N per m, chosen on the ideal model. Three things still go wrong on the real robot:

1. **The gains.** On a 1° ramp, gravity pulls the 8 kg robot back with 1.37 N. With those gains, the speed drops by 0.054 m/s and takes 5.2 s to come back within 0.005 m/s of 0.5 m/s. Larger gains would recover faster, but how large is safe? The ideal model says any gain is safe; the real robot rings when the gain is too high.
2. **The command step.** A heading correction of 0.1 rad (5.7°) makes RO-060's heading PID ask for 2.85 N · m in one sample, 14 times the 0.2 N · m the motors can give, and the heading overshoots (G-2983), passes the command before it settles, by 21 percent.
3. **The sensor noise.** While the robot drives straight, every wheel-encoder tick makes the PID's derivative action jump by 0.043 N · m, a fifth of the torque limit. A gyro has no ticks, but it carries the frame's 10 Hz shaking into the derivative action as ±0.085 N · m.

Figure 1 draws the PID loop this Note tunes, with each change marked where it acts.

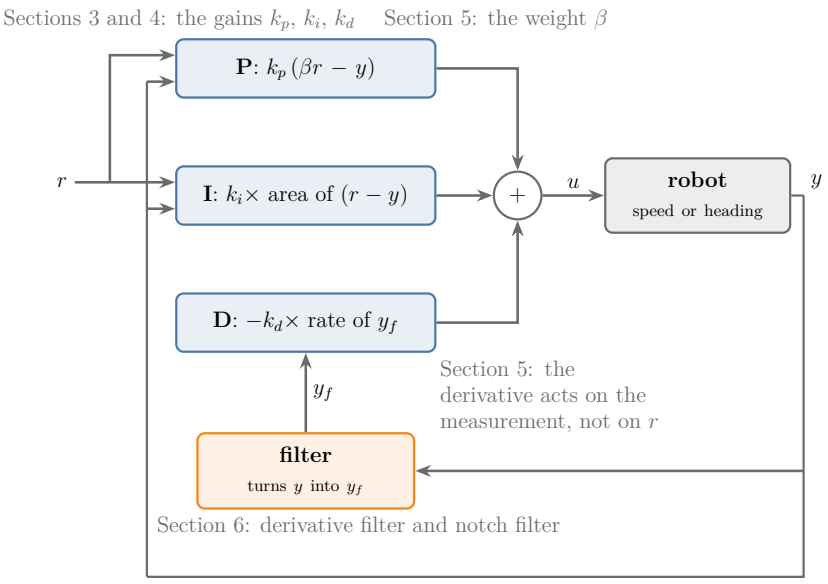


Figure 1: The PID loop of this Note. The reference r and the measurement y feed the P action, k_p times (beta r minus y), and the I action, k_i times the area of (r minus y); the D action, minus k_d times the rate of y_f, sees only the filtered measurement y_f. The three add up to the command u that drives the robot (its speed or its heading). Notes in grey name the section that sets each part: the gains (Sections 3 and 4), the weight beta and the derivative on the measurement (Section 5), the derivative filter and the notch filter (Section 6)

The sections answer the three problems in this order:

- which three numbers a PID controller has, and the two standard ways to write them (Section 2);
- the **Ziegler–Nichols method** (G-3093): raise the gain until the speed loop oscillates, then use the oscillation's gain and period (Section 3);
- **model-based tuning** (G-3097): measure one bump test, fit three numbers, and use a rule built for that model (Section 4);
- the **derivative kick** (G-3098) and **set-point weighting** (G-3100), which stop a command step from jerking the motors (Section 5);
- the **derivative filter** (G-3101) and the **notch filter** (G-3102), which keep the encoder steps and the 10 Hz shaking out of the derivative action (Section 6).

2. The PID forms: which numbers we tune

> **Key point:** Tuning starts by naming the numbers to tune. The same **ideal PID controller** (G-3090) can be written with three gains (the **parallel form**, G-3091) or with one gain and two times (the **standard form**, G-3092); the two times compare directly with how fast the robot moves.

2.1 The ideal PID controller in parallel form

All three problems of the Overview are about the numbers inside the PID controller, so we first write them down. The PID controller (G-2965) looks at the error, the command minus the measurement, and adds three actions: one proportional to the error, one to the area under the error so far, one to the error's rate of change, its derivative (G-595). Written with one gain per action, this is the **ideal PID controller** (G-3090) in its **parallel form** (G-3091) (FBS §11.1, eq. 11.1). "Ideal" means the textbook law with nothing added: no limits and no filters. Sections 5 and 6 add what a real robot needs.

For the robot's heading loop, which turns the robot by a torque, with RO-060's gains:

- e is the error, the heading command minus the measured heading, in rad;
- u is the controller's output, the torque it asks the motors for, in N · m (the motors give at most 0.2 N · m either way);
- k_p is the proportional gain (G-2959), 0.2 N · m per rad;
- k_i is the integral gain (G-2964), 0.05 N · m per rad · s, which multiplies the area under the error;
- k_d is the derivative gain (G-2961), 0.283 N · m · s per rad, which multiplies the error's rate of change.

$$u = k_p\, e + k_i \int_0^t e\, dt + k_d \frac{de}{dt}$$

The integral sign, $\int$, with 0 below and t above, means the area under the error curve from time 0 to now, t. An error of 0.1 rad held for 2 s has the area:

$$0.1 \times 2 = 0.2 \text{ rad} \cdot \text{s}$$

Figure 2 (top) draws this form: the error goes down three branches, each with its own gain, and the three results add up.

2.2 The standard form: one gain and two times

Three separate gains hide how the actions compare in time. The same controller can be written with one overall gain and two times (FBS §11.1):

- k_p is the same proportional gain, 0.2 N · m per rad, now multiplying all three actions;
- T_i is the **integral time** (G-3103), 4 s for the heading PID;
- T_d is the **derivative time** (G-3104), 1.415 s for the heading PID.

$$u = k_p\left(e + \frac{1}{T_i}\int_0^t e\, dt + T_d \frac{de}{dt}\right)$$

The form with one gain and two times is the **standard form** (G-3092) (FBS §11.1), Figure 2 (bottom). Multiplying out the bracket gives back the parallel form, so the two gains of the parallel form are, our numbers first:

$$k_i = 0.2/4 = 0.05$$

$$k_d = 0.2 \times 1.415 = 0.283$$

and in symbols:

$$k_i = k_p/T_i$$

$$k_d = k_p\, T_d$$

The speed PI controller has no derivative action; with its gains of 20 and 10, its integral time is:

$$T_i = 20/10 = 2 \text{ s}$$

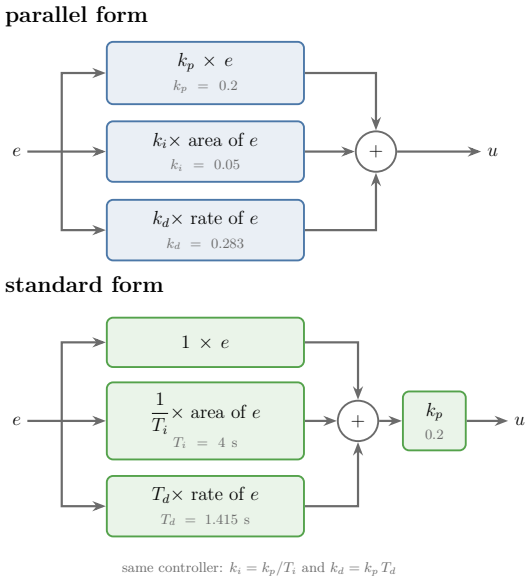


Figure 2: One ideal PID controller drawn two ways, with the heading PID's numbers. Top, parallel form: the error e goes through three blocks, k_p times e (k_p = 0.2), k_i times the area of e (k_i = 0.05) and k_d times the rate of e (k_d = 0.283), and the three add up to u. Bottom, standard form: the three blocks are 1 times e, 1/T_i times the area of e (T_i = 4 s) and T_d times the rate of e (T_d = 1.415 s); their sum is multiplied by one gain, k_p = 0.2. The note at the bottom states the link: k_i = k_p/T_i and k_d = k_p T_d

What the two times mean. Each time is the time over which one action catches up with the proportional action.

The integral time. Hold a speed error of 0.1 m/s on the speed PI controller. The proportional action and the integral action's growth per second are:

$$20 \times 0.1 = 2 \text{ N}$$

$$10 \times 0.1 = 1 \text{ N per s}$$

After $T_i = 2$ s the integral action has added:

$$1 \times 2 = 2 \text{ N}$$

as much as the proportional action. A short integral time means the integral action takes over quickly.

The derivative time. Let the heading error fall steadily by 0.1 rad/s. The derivative action is:

$$0.283 \times (-0.1) = -0.0283 \text{ N} \cdot \text{m}$$

In $T_d = 1.415$ s the error falls by 0.1415 rad, which changes the proportional action by:

$$0.2 \times (-0.1415) = -0.0283 \text{ N} \cdot \text{m}$$

the same amount. The derivative action applies now the change the proportional action will see T_d seconds later, a prediction by a straight line (FBS §11.1).

Both times are measured in seconds, so they compare directly with the robot's own times, such as the speed plant's time constant of 4 s. Changing k_p alone scales all three actions together and keeps their timing. The PX4 tuning guide gives this as the reason for the standard form: a new drone can start from the gains of a similar one and adjust the one overall gain (PX4 tuning guide).

In real stacks. PX4's rate controller offers both forms at once: its parameters MC_ROLLRATE_P, I, D and an overall gain MC_ROLLRATE_K; setting K = 1 gives the parallel form, setting P = 1 the standard form (PX4 tuning guide). The PID class of ros2_control (`pid.hpp` in `control_toolbox`) and the speed controllers of Autoware and Apollo take three parallel gains named p, i, d or kp, ki, kd. The numbers are now named, but nothing yet says which values to give them.

3. Ziegler–Nichols: tuning from one experiment

> **Key point:** The Ziegler–Nichols method raises the proportional gain until the loop oscillates steadily, reads that **critical gain** and **critical period** (G-3094), and turns them into PID gains by a table. On the robot's speed loop it gives fast but fragile gains.

3.1 Why the ideal speed model never oscillates

The first problem of the Overview asks how large the speed gains can be before the robot rings. On the ideal model the answer is: any size. Under proportional control alone, the speed plant of RO-059 is a first-order system. With:

- m the robot's mass, 8 kg;
- b the rolling drag, 2 N · s/m;
- v the speed, in m/s, and r the commanded speed, 0.5 m/s;
- k_p the proportional gain, in N per m/s;

the drive force, the gain times the speed error, minus the drag accelerates the mass:

$$m\,\frac{dv}{dt} = k_p(r - v) - b\,v$$

$$m\,\frac{dv}{dt} = k_p\,r - (k_p + b)\,v$$

The second line has the shape of a first-order system (G-2987): the speed approaches its final value along a curve that never overshoots, with the time constant (G-2988):

$$\frac{m}{k_p + b}$$

For two gains:

$$k_p = 20: \quad 8/22 = 0.36 \text{ s}$$

$$k_p = 200: \quad 8/202 = 0.040 \text{ s}$$

A larger gain only makes the curve steeper. The heading plant is no better guide: with proportional control alone nothing damps its turn, its damping ratio (G-2992), the number that sets how fast a swing dies out, is 0 at every gain, so it swings forever and only the period changes; no single gain stands out. A tuning experiment needs a loop that is calm at a low gain and starts to oscillate only above some gain. The real speed loop is such a loop, because of a detail the ideal model left out.

3.2 The measured speed arrives late

The controller does not see the true speed. The wheel encoders count ticks, each 0.307 mm of wheel travel (Section 6.1 works it out). Within one sample of 0.01 s, one tick more or less changes the computed speed by:

$$0.000307/0.01 = 0.031 \text{ m/s}$$

A jump of 0.031 m/s is 6 percent of 0.5 m/s, so the speed computed from one sample is jumpy. The controller therefore uses the moving average (G-2267) of the last 20 samples, 0.2 s of wheel speeds, where one tick changes the average by only:

$$0.000307/0.2 = 0.0015 \text{ m/s}$$

ros2_control's diff_drive_controller does the same with a rolling mean of the last 10 samples by default (its parameter `velocity_rolling_window_size`). Figure 3 slides that 0.2 s window along the speed while the robot speeds up at 0.25 m/s^2. Watch the orange dot, the average the controller reads now: it always sits level with the true speed of a moment earlier.

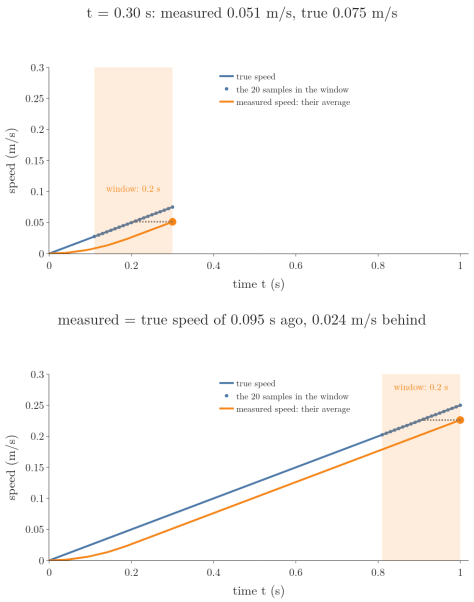


Figure 3: The robot speeds up at 0.25 m/sˆ2 (blue line, the true speed). Each frame slides the 0.2 s window (orange band) one step on; the 20 samples inside it are the blue dots, and their average is the orange dot, which the controller reads at that moment. The dotted line runs back from the orange dot to the time when the true speed had that value. The orange line, the measured speed, starts at 0 and then runs parallel to the true speed, 0.024 m/s below it. The last frame states it: the measured speed is the true speed of 0.095 s ago

Why the average is late. The window holds the speeds at the times t, $t-0.01$, …, $t-0.19$

s. While the speed rises along a straight line, the average of the 20 samples is the speed at the window's middle. With:

- n the number of samples in the window, 20;
- h the sample period (G-3106), the time between two readings, 0.01 s;

the middle lies this far back, our numbers first:

$$\text{lag} = \frac{20 - 1}{2} \times 0.01$$

$$= 0.095 \text{ s}$$

and in symbols:

$$\text{lag} = \frac{n - 1}{2} h$$

At 0.25 m/s^2, 0.095 s of lag is a speed gap of:

$$0.25 \times 0.095 = 0.024 \text{ m/s}$$

So the controller acts on a speed about 0.1 s old, like a driver steering by where the car was a moment ago. A fast correction, made on old news, overshoots, and a high gain makes fast corrections. The experiment of Section 3.3 finds the gain at which this turns into a steady oscillation.

3.3 Raise the gain until the loop oscillates

Ziegler and Nichols's frequency-response experiment works on the running loop (FBS §11.3): turn off the integral and derivative actions, then raise the proportional gain step by step until the output oscillates steadily. We run it on the robot cruising at 0.5 m/s, with the speed loop of RO-059: the drive force is the 1 N that holds 0.5 m/s against the drag, plus k_p times the error, and k_p is raised from its 20 N per m/s. At t = 0.2 s the speed command steps up by only 0.01 m/s, so that even at a gain of 200 the force stays inside the ±4 N limit (between −0.54 and 3.0 N); a saturated drive would spoil the reading.

Figure 4 plays four runs. Watch the ringing after the step last longer with each larger gain, until at 200 N per m/s it neither grows nor dies.

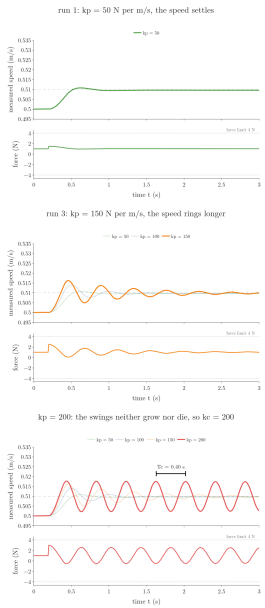


Figure 4: The Ziegler–Nichols experiment on the speed loop, one run per gain: k_p = 50, 100, 150 and 200 N per m/s, proportional control only, the command stepping from 0.5 to 0.51 m/s (dashed line) at t = 0.2 s. Each run plays in steps and stays faint once the next starts. Top: the measured speed; bottom: the drive force, with the ±4 N limit dotted. At 50 the speed settles; at 100 and 150 it rings, longer at 150; at 200 the swings keep the same size. The last frame marks two neighbouring peaks of the 200 run, one critical period T_c = 0.40 s apart

The gain at which the oscillation neither grows nor dies is the **critical gain**, and its period is the **critical period** (G-3094) (FBS §11.3), here:

- k_c is the critical gain, 200 N per m/s;
- T_c is the critical period, 0.40 s.

At a gain of 220 the swings grow from one period to the next (notebook): above k_c the loop is unstable.

Why it oscillates there. A steady oscillation goes round the loop and comes back the same size and in step with itself. The sign flip of the error, $r - y$, already turns it upside down once, so the drive and the averaging together must delay it by half a period, 180°, and leave its size unchanged. At the critical frequency (one swing per 0.396 s, read as 0.40 s in Figure 4) the delays are (notebook):

- the drive's own lag, the plant's response to a force swing: 89.1°;
- the 20-sample average, 0.095 s late: 86.4°;
- the controller's 0.01 s steps, which act like a delay of half a sample, 0.005 s: the remaining 4.5°.

At the same frequency the drive and the average shrink a swing of the force to 1/200.7 of its size in the measured speed, so a gain of 200.7 brings it back to exactly its starting size;

Figure 4 reads it as 200. In the language of the Nyquist criterion (G-3041), the loop's curve passes through the critical point (G-3036), -1, at this frequency (FBS §11.3).

3.4 The Ziegler–Nichols rules and their price

Ziegler and Nichols turned the two numbers into controller settings by a table (FBS Table 11.1b). They built it in the 1940s from simulations of many processes, each tuned by hand (FBS §11.3):

Controller	k_p	T_i	T_d
P	$0.5\,k_c$		
PI	$0.45\,k_c$	$T_c/1.2$	
PID	$0.6\,k_c$	$T_c/2$	$T_c/8$

The speed loop is RO-060's PI controller, so with $k_c = 200$ N per m/s and $T_c = 0.40$ s:

$$k_p = 0.45 \times 200 = 90$$

$$T_i = 0.40/1.2 = 0.333 \text{ s}$$

$$k_i = 90/0.333 = 270$$

in N per m/s and N per m, 4.5 and 27 times the speed PI's 20 and 10. Figure 6 in Section 4.3 runs them on the 1° ramp: the speed drops by only 0.021 m/s and is back within 0.005 m/s after 0.8 s. The price shows elsewhere:

- a step of the speed command by 0.05 m/s overshoots by 83 percent and drives the force to its 4 N limit;
- the loop has a phase margin (G-3045), the extra phase lag it can take at its crossover before it oscillates, of only 18°, and its delay margin (G-3046), the extra delay it can take, is 0.032 s; with the speed measured 0.04 s later still, it oscillates (Figure 6, bottom).

Feedback Systems names the two drawbacks of the rules: they use too little information about the process, and the loops they give lack robustness (FBS §11.3). The experiment saw only one point of the loop, the 180° point. The next section measures the whole response once and lets a rule use more of it.

4. Model-based tuning: a bump test and a small model

> **Key point:** One **bump test** (G-3095), a step of the drive force with the loop open, gives three numbers: a gain, a delay and a time constant, the **first-order-plus-time-delay model** (G-3096). A tuning rule built for that model gives speed gains that cut the ramp's speed drop from 0.054 to 0.039 m/s and keep a delay margin of 0.27 s.

4.1 The bump test

The Ziegler–Nichols gains were fast but fragile, because one oscillation is little information. A bump test records the robot's whole response to one step (FBS §11.3): with the controller off, the drive force steps from 0 to 1 N and stays there, and we record the measured speed until it settles. Figure 5 plays the test. Watch the top panel for the final speed and the bottom panel, the first second enlarged, for the late start.

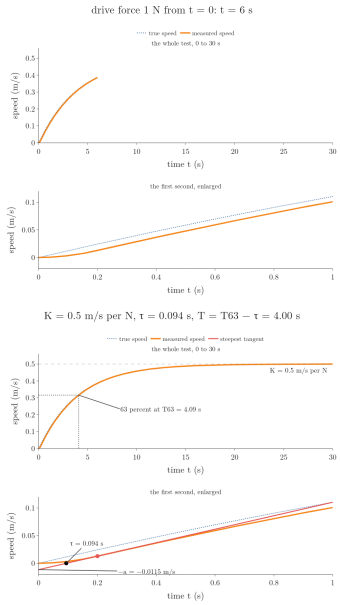


Figure 5: The bump test. The robot stands still; at t = 0 the drive force steps to 1 N and stays there. Top, the whole test over 30 s: the measured speed (orange) rises along the true speed (blue dots) and settles at 0.5 m/s. Bottom, the first second enlarged: the measured speed starts late. The later frames read the test: the steepest tangent (red, 0.122 m/sˆ2 at t = 0.2 s) crosses the time axis at tau = 0.094 s and the speed axis at −a = −0.0115 m/s; the speed settles at K = 0.5 m/s per N; it reaches 63 percent of that at T63 = 4.09 s. The last frame holds K = 0.5 m/s per N, tau = 0.094 s and T = T63 − tau = 4.00 s

Three readings come off the test (FBS §11.3, Fig. 11.7a):

- K is the gain, the final speed per newton of drive force: 0.5 m/s per N;
- τ is the apparent delay, where the steepest tangent of the response crosses the time axis: 0.094 s;
- T_{63} is the time at which the speed has covered 63 percent of its rise: 4.09 s.

The 63 percent mark is one time constant (G-2988) of a first-order system. Here the response starts late by τ, so the time constant T is:

$$T = 4.09 - 0.094 = 4.00 \text{ s}$$

$$T = T_{63} - \tau$$

The three numbers agree with the physics of the robot: the gain K is one over the drag, 0.5 m/s per N; the time constant T is the mass over the drag, 4 s; the delay τ is close to the averaging lag of Section 3.2, 0.095 s. The test needs patience, though: the speed must settle before K can be read, 30 s here, where the Ziegler–Nichols experiment needed a few seconds per gain (FBS §11.3).

4.2 The model and the tuning rule

The three numbers describe the response as a delay followed by a first-order rise, the **first-order-plus-time-delay (FOTD) model** (G-3096) (FBS eq. 11.9), also called first order plus dead time. With:

- F the size of the force step, 1 N;
- t the time since the step, in s;
- Δy the rise of the measured speed, in m/s;
- K, τ and T the gain, the delay and the time constant read off the bump test;
- exp the exponential function, $\exp(x) = 2.718^x$, so that $\exp(-1) = 0.368$;

the model says:

$$\Delta y = 0 \quad \text{for} \quad t < \tau$$

$$\Delta y = KF\Big(1 - \exp(-\tfrac{t-\tau}{T})\Big)$$

$$\text{for} \quad t \geq \tau$$

One time constant after the delay, the bracket is:

$$1 - 0.368 = 0.632$$

the 63 percent mark of Section 4.1.

A delay that is small next to the time constant, as here (0.094 s against 4.00 s), makes a process lag dominated (FBS §11.3). For such a process, rules based on this model suit PI controllers; a PID needs a different method (FBS §11.3), and the speed loop is a PI controller anyway.

Tuning a controller from a model of the process is **model-based tuning** (G-3097). Feedback Systems gives PI rules for the FOTD model that use all three numbers (FBS eq. 11.10a). With $K = 0.5$ m/s per N, $\tau = 0.094$ s and $T = 4.00$ s, the proportional gain, our numbers first:

$$0.15 \times 0.094 = 0.0141$$

$$0.35 \times 4.00 = 1.4$$

$$0.5 \times 0.094 = 0.047$$

$$k_p = (0.0141 + 1.4)/0.047$$

$$= 1.4141/0.047 = 30.1$$

and in symbols:

$$k_p = \frac{0.15\,\tau + 0.35\,T}{K\tau}$$

The integral gain, our numbers first:

$$0.46 \times 0.094 = 0.04324$$

$$0.02 \times 4.00 = 0.08$$

$$0.5 \times 0.094^2 = 0.004418$$

$$k_i = (0.04324 + 0.08)/0.004418$$

$$= 0.12324/0.004418 = 27.9$$

and in symbols:

$$k_i = \frac{0.46\,\tau + 0.02\,T}{K\tau^2}$$

The units work out to N per m/s and N per m. The rule's proportional gain is a third of Ziegler–Nichols's 90; Feedback Systems notes that such improved rules give lower gains than the original table (FBS §11.3).

The same test can also feed Ziegler and Nichols's other experiment, their step-response method (FBS Table 11.1a). The step-response method uses only τ and a, where the steepest tangent crosses the speed axis, 0.0115 m/s here. Its PI gains:

$$k_p = 0.9/0.0115 = 78.3$$

$$T_i = 0.094/0.3 = 0.313 \text{ s}$$

$$k_i = 78.3/0.313 = 250$$

close to the frequency method's 90 and 270, and just as fragile.

4.3 Three tunings side by side

Three pairs of gains now stand for the speed loop: RO-060's speed PI, the bump-test rule's and Ziegler–Nichols's. Figure 6 drives all three onto the 1° ramp of the Overview. Watch the top panel for how far each speed drops and how fast it returns, then the bottom panel, where the measured speed arrives 0.04 s later still.

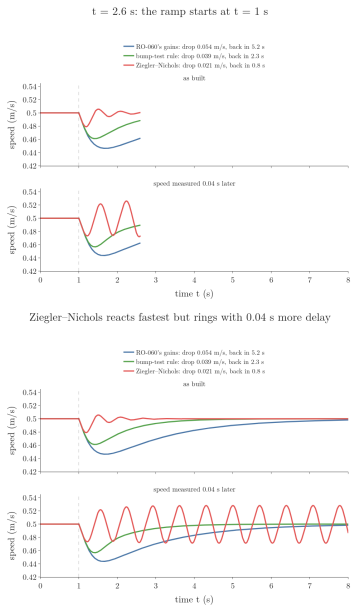


Figure 6: Three tunings of the speed PI against the same load: at t = 1 s (dashed line) the robot, cruising at 0.5 m/s, meets a 1° ramp, and gravity pulls back with 1.37 N. Blue: RO-060's gains, k_p = 20 and k_i = 10. Green: the bump-test rule, k_p = 30.1 and k_i = 27.9. Red: Ziegler–Nichols, k_p = 90 and k_i = 270. Time runs frame by frame over 8 s. Top, the loop as built: the speeds drop by 0.054, 0.039 and 0.021 m/s and are back within 0.005 m/s after 5.2, 2.3 and 0.8 s. Bottom, the same loops with the measured speed 0.04 s later: the blue and green curves barely change, the red one swings between 0.471 and 0.528 m/s and keeps swinging

Gains	k_p	k_i	Speed drop on the ramp	Back within 0.005 m/s
RO-060's gains	20	10	0.054 m/s	5.2 s
bump-test rule	30.1	27.9	0.039 m/s	2.3 s
Ziegler–Nichols	90	270	0.021 m/s	0.8 s

Gains	Phase margin	Delay margin	Overshoot, 0.05 m/s step
RO-060's gains	70°	0.49 s	6.2 percent
bump-test rule	58°	0.27 s	14.6 percent
Ziegler–Nichols	18°	0.032 s	83 percent

Gains are in N per m/s and N per m; the phase margin (G-3045) and the delay margin (G-3046), how much extra lag and extra delay the loop tolerates (stability margins), are computed from the loop at 100 Hz (notebook).

The bump-test gains answer the first problem of the Overview. They cut the speed drop on the 1° ramp from 0.054 to 0.039 m/s and the recovery from 5.2 to 2.3 s, and they still

tolerate 0.27 s of extra delay. Ziegler–Nichols recovers fastest, but 0.04 s of extra delay makes it oscillate. RO-060's gains are the calmest and the slowest. Which pair to keep depends on what the robot needs, and the model-based rule sits in the useful middle. The step overshoot of every pair can still be lowered without touching its gains, by the weighting of Section 5.3.

In real stacks. The PX4 tuning guide tunes a drone by hand: it raises a gain by 20 to 30 percent per try, then by 5 to 10 percent for the final touches; it tunes the proportional and derivative gains first and the integral gain last; and it starts with the rate controller, the innermost loop, because that loop "affects all flight modes" (PX4 tuning guide). Tuning the inner loop of a cascade first is the order Feedback Systems gives too (FBS §1.5). The speed loops of Autoware and Apollo are PI controllers too: their shipped parameters set the derivative gain to 0 (Autoware's longitudinal controller: kp 1.0, ki 0.1, kd 0.0; Apollo's `controller_conf.pb.txt`: kp 2.0, ki 0.3, kd 0.0 at low speed; files listed in Sources). The speed loop is now tuned; the heading loop, which needs its derivative action, still kicks.

5. Derivative kick and set-point weighting

> **Key point:** When the command jumps, the error jumps, and the derivative of a jump is huge: the **derivative kick** (G-3098). Taking the derivative of the measurement instead, the **derivative on measurement** (G-3099), removes it; **set-point weighting** (G-3100) also lowers the proportional jump and cuts the heading's overshoot from 22.5 to 1.6 percent.

5.1 The heading PID and its test

The second problem of the Overview is the heading loop. The heading plant has no friction to calm a turn, so its controller needs the derivative action (G-2960), which brakes the turn before the target. We use RO-060's heading PID, the gains of Section 2: $k_p = 0.2$ N · m per rad, $k_i = 0.05$ N · m per rad · s, $k_d = 0.283$ N · m · s per rad; RO-060 built it from its PD controller (G-2962) by adding the error up, the integral action (G-2963). With the yaw inertia of 0.1 kg · m^2, these gains give the closed loop one pole (G-3000), a rate at which a part of the response dies away, at -1.93 rad/s and a complex pair at -0.45 ± 0.24i rad/s (notebook): a gentle turn that takes several seconds to settle.

The test is a heading correction from the path tracker: at t = 0.5 s, the heading command steps from 0 to 0.1 rad (5.7°). The step is small, so the proportional action alone stays inside the 0.2 N · m limit:

$$0.2 \times 0.1 = 0.02 \text{ N} \cdot \text{m}$$

Until Section 6 the measured heading is exact.

5.2 Derivative kick

The derivative action of Section 2.1 acts on the rate of change of the error. In the sample where the command steps, the error jumps by 0.1 rad within one 0.01 s sample. With:

- Δe the error's jump, 0.1 rad;
- h the sample period, 0.01 s;
- k_d the derivative gain, 0.283 N · m · s per rad;

the computed rate and the derivative action in that sample are:

$$\frac{\Delta e}{h} = \frac{0.1}{0.01} = 10 \text{ rad/s}$$

$$k_d \times 10 = 0.283 \times 10 = 2.83 \text{ N} \cdot \text{m}$$

With the proportional action, the request is:

$$2.83 + 0.02 = 2.85 \text{ N} \cdot \text{m}$$

$$2.85/0.2 = 14.25$$

14 times what the motors can give. A shorter sample would make it larger still; an ideal derivative of a jump is infinite (Beauregard). The spike is the **derivative kick** (G-3098) (Beauregard; FBS §11.5).

The fix. Between jumps the command stays put, so the rate of change of the error e is minus the rate of change of the measurement. With:

- r the heading command, in rad;
- y the measured heading, in rad;

$$\frac{de}{dt} = \frac{dr}{dt} - \frac{dy}{dt}$$

$$= -\frac{dy}{dt} \quad \text{while } r \text{ is constant}$$

So a derivative action computed from the measurement,

$$-k_d \frac{dy}{dt}$$

equals the old one at every moment except the jump, where it ignores the command. Taking the derivative of the measurement instead of the error is the **derivative on measurement** (G-3099) (Beauregard; FBS §11.5). Figure 7 turns the robot with both. Watch the torque request at t = 0.5 s, then the two headings below.

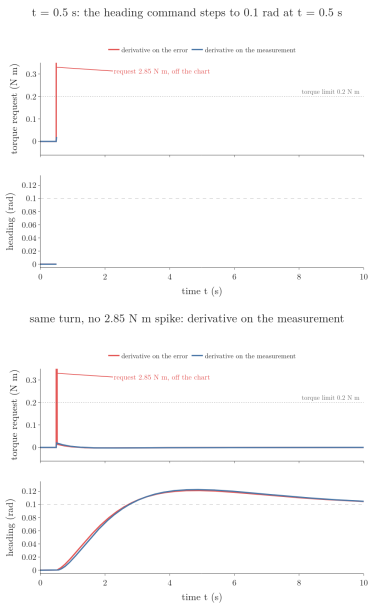


Figure 7: Derivative kick on RO-060's heading PID (k_p = 0.2, k_i = 0.05, k_d = 0.283). At t = 0.5 s the heading command steps from 0 to 0.1 rad (dashed line, bottom). Top: the torque request, with the 0.2 N m limit dotted. Red, derivative on the error: in the step's sample the request spikes to 2.85 N m, far off the chart. Blue, derivative on the measurement: the request starts at 0.02 N m. Bottom: the two headings lie almost on top of each other (overshoot 21.1 and 22.5 percent). The last frame holds the whole 10 s

The heading barely changes (21.1 against 22.5 percent overshoot), because the motors clip the spike to 0.2 N · m for one sample. The derivative on measurement turns the robot the same way without sending the motors a 2.85 N · m command at every heading step (Beauregard). The overshoot, though, is still 22.5 percent.

5.3 Set-point weighting

The proportional action also reacts to the step: it jumps from 0 to 0.02 N · m at once. Feedback Systems lets each action see only a fraction of the command (FBS eq. 11.15):

- β is the weight of the command in the proportional action, from 0 to 1;
- γ is the weight of the command in the derivative action, normally 0 (FBS §11.5);
- k_p, k_i, k_d, r and y are as above.

$$u = k_p(\beta r - y)$$

$$+k_i \int_0^t (r - y)\, dt$$

$$+k_d\left(\gamma \frac{dr}{dt} - \frac{dy}{dt}\right)$$

Letting each action see its own fraction of the command is **set-point weighting** (G-3100) (FBS §11.5). With $\gamma = 0$ it contains the derivative on measurement of Section 5.2. The integral action keeps the full error e: a steady state needs the area to stop growing, which happens only when the measured heading equals the command, so the heading still ends on the command (FBS §11.5). The weights change only how the loop answers the command; its answer to a push or to sensor noise stays the same (FBS §11.5). Figure 8 turns the robot with $\beta = 1$, 0.5 and 0. Watch the overshoot shrink as β falls, and the turn slow down at $\beta = 0$.

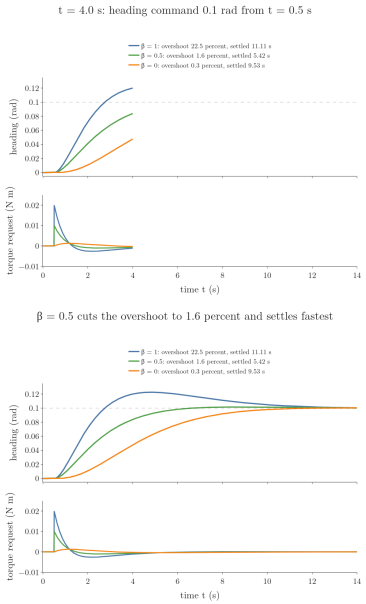


Figure 8: Set-point weighting on RO-060's heading PID (derivative on the measurement). The heading command steps from 0 to 0.1 rad at t = 0.5 s (dashed line). Top: the heading for beta = 1 (blue), 0.5 (green) and 0 (orange). Bottom: the torque request, which jumps at the step by 0.02, 0.01 and 0 N m. Overshoot 22.5, 1.6 and 0.3 percent; within 2 percent of the command 11.11, 5.42 and 9.53 s after the step. Every run ends on 0.1 rad. The last frame holds all 14 s

β	Jump of the request at the step	Overshoot	Within 2 percent after
1	0.02 N · m	22.5 percent	11.11 s
0.5	0.01 N · m	1.6 percent	5.42 s
0	0 N · m	0.3 percent	9.53 s

With $\beta = 0.5$ the heading reaches 0.1 rad with only 1.6 percent overshoot and settles fastest; with $\beta = 0$ only the integral action drives the turn, which is smooth but slow. Feedback Systems shows the same on a car's cruise control: $\beta = 1$ overshoots and nearly saturates the throttle, $\beta = 0$ does not overshoot (FBS §11.5, Fig. 11.13). The two changes answer the

second problem of the Overview: derivative on measurement removes the 2.85 N · m kick, and $\beta = 0.5$ cuts the overshoot from 22.5 to 1.6 percent.

In real stacks. PX4's rate controller puts the derivative action on the feedback path "in order to avoid an effect known as the derivative kick" (PX4 tuning guide). Its code, `rate_control.cpp`, computes the torque as the proportional action on the rate error, plus the integral, minus the D gain times the measured angular acceleration, plus a feedforward of the rate command. The ros2_control PID, in `control_toolbox`'s `pid.cpp`, computes the derivative from the error, (error − last error)/dt, unless the caller passes the error's rate itself, so a jump in the command kicks unless the caller supplies $-dy/dt$. Apollo's PID does the same, `diff = (error - previous_error_) / dt` in its `pid_controller.cc`, with the derivative gain 0 in the shipped speed controller. The command no longer kicks; the measurement still does.

6. Filters inside the loop

> **Key point:** The derivative action amplifies whatever jumps in the measurement. A **derivative filter** (G-3101) smooths it at the cost of some lag, and a **notch filter** (G-3102) removes one known frequency, such as the frame's 10 Hz shaking, with almost no lag elsewhere.

6.1 The derivative filter

The third problem of the Overview comes from the measurement itself. Section 5 took the heading as exact; the robot measures it with its wheel encoders. With:

- r_w the wheel radius, 0.05 m, of the differential-drive robot;
- N_e the encoder's ticks per wheel turn, 1024;
- L the distance between the wheels, 0.2 m;
- h the sample period, 0.01 s;
- k_d the derivative gain, 0.283 N · m · s per rad, and y the measured heading, in rad;

one tick of one wheel is a travel of:

$$2\pi \times 0.05/1024 = 0.3068 \text{ mm}$$

$$2\pi\, r_w/N_e$$

which turns the measured heading by:

$$0.0003068/0.2 = 1.534 \text{ mrad}$$

Driving straight at 0.5 m/s, the two wheels tick at slightly different moments, so the measured heading flickers by one tick, 1.53 mrad, while the true heading hardly moves. The derivative on measurement turns one tick within one sample into a rate and a derivative action of:

$$0.00153/0.01 = 0.153 \text{ rad/s}$$

$$0.283 \times 0.153 = 0.043 \text{ N} \cdot \text{m}$$

a fifth of the 0.2 N · m limit, in one sample, and back again at the next tick. The motors receive a train of torque spikes for no real motion.

The fix smooths the rate before it is used, with a low-pass filter: a filter that lets slow changes through and smooths fast ones. A first-order low-pass with time constant T_f covers 63 percent of a step of its input in T_f seconds. Put on the derivative path, it is the **derivative filter** (G-3101) (FBS §11.5). With:

- D the derivative action after the filter, in N · m;
- T_f the filter's time constant, in s;
- N a number between 5 and 20 that sets T_f (FBS §11.5);
- T_d the derivative time of Section 2.2, 1.415 s;

the filtered derivative action follows the raw one with the lag T_f (FBS §11.5):

$$T_f \frac{dD}{dt} + D = -k_d \frac{dy}{dt}$$

$$T_f = T_d/N$$

With $N = 10$:

$$T_f = 1.415/10 = 0.1415 \text{ s}$$

A tick lasts one sample, 0.01 s, a fourteenth of T_f, so the filter lets through only a small part of it. As a transfer function (G-2999), the ideal derivative is k_d times s, and the filtered one (FBS §11.5) is:

$$\frac{k_d\, s}{1 + s\, T_f}$$

Figure 9 drives the robot straight for one second. Watch each red spike of the unfiltered derivative action and the small blue bump the filter makes of it.

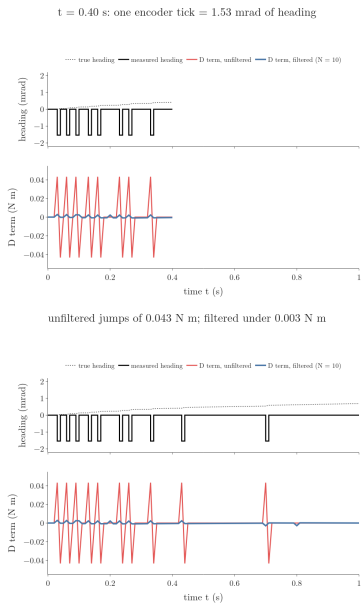


Figure 9: The derivative filter. The robot drives straight at 0.5 m/s and the heading PID holds the heading at 0. Top: the measured heading in mrad (black), which flickers by one encoder tick, 1.53 mrad, while the true heading (grey dots) rises to only 0.72 mrad. Bottom: the derivative action. Red, unfiltered: every tick makes a spike of 0.043 N m up and down. Blue, filtered with the time constant T_f = 0.1415 s (N = 10): it stays under 0.003 N m. The torque limit, 0.2 N m, lies far above the chart. Time runs frame by frame over 1 s

The filter is not free: it delays the derivative action, and the derivative action is what gives the heading loop its phase margin (G-3045), the extra lag it can take before it oscillates. The choice of N trades the two (notebook):

N	T_f	Swing of the derivative action	Phase margin
no filter	0	± 0.043 N · m	75°
20	0.071 s	± 0.0054 N · m	64°
10	0.14 s	± 0.0030 N · m	54°
5	0.28 s	± 0.0016 N · m	39°

With $N = 10$ the spikes shrink from 0.043 to under 0.003 N · m, and 54° of phase margin remains. The encoder ticks are handled; the faster, regular disturbance of the next section needs a different filter.

6.2 The notch filter

A gyro measures the turn rate directly, so the derivative action can take minus the derivative gain k_d, 0.283 N · m · s per rad, times the gyro's reading: no differencing, and no encoder ticks. PX4's drones feed their turn-rate control from the gyro in the same way (PX4 controller diagrams). But the robot's frame bends back and forth at 10 Hz when the motors shake it,

and the gyro sees that bending as a turn rate of ± 0.3 rad/s. The derivative action passes it on:

$$0.283 \times 0.3 = 0.085 \text{ N} \cdot \text{m}$$

a 10 Hz torque swing of over 40 percent of the 0.2 N · m limit. Motors driven like this run hot (PX4 filter tuning).

A low-pass filter strong enough for this must sit far below 10 Hz. The heading loop does its work around its gain crossover frequency (G-3042), 0.46 Hz, where its loop gain is 1. A low-pass at 1 Hz lets through a tenth of the 10 Hz shaking but lags 24° at 0.46 Hz, and the phase margin falls from 75° to 52°. PX4's documentation makes the same point: without the notch filter, "you'd have to set the low pass filter cutoff much lower (increasing the phase lag)" (PX4 filter tuning).

The shaking has one frequency, so a filter can remove that frequency alone, the way a good ear tunes out the hum of a fridge and still hears every word. A filter that blocks a narrow band around one frequency and passes the rest is a **notch filter** (G-3102) (PX4 filter tuning). With:

- ω_0 the angular frequency of the shaking, in rad/s;
- Q the sharpness of the notch, the centre frequency over the width of the band;
- s the variable of the transfer function (G-2999), which becomes $i\omega$ for a wave of angular frequency ω in the frequency response (G-3020).

For the 10 Hz shaking and a band 5 Hz wide:

$$\omega_0 = 2\pi \times 10 = 62.8 \text{ rad/s}$$

$$Q = 10/5 = 2$$

The notch is:

$$F(s) = \frac{s^2 + \omega_0^2}{s^2 + (\omega_0/Q)\, s + \omega_0^2}$$

Checks at easy frequencies. For a wave of angular frequency ω, the square s^2 becomes minus ω squared. A steady signal has $\omega = 0$, so every s term vanishes:

$$F = \frac{\omega_0^2}{\omega_0^2} = 1$$

The steady part passes unchanged. The shaking has $\omega = \omega_0$, and the top becomes:

$$-\omega_0^2 + \omega_0^2 = 0$$

so the 10 Hz shaking is blocked completely. For a very fast wave the s^2 terms outgrow the rest, top and bottom alike, so F is close to 1 again.

Figure 10 draws both filters as frequency responses, the gain and the phase against frequency on log axes, as in the Bode plot (G-3023). Read the two dashed lines: at 0.46 Hz the low-pass already lags 24° and the notch 1.4°; at 10 Hz the low-pass passes a tenth and the notch nothing.

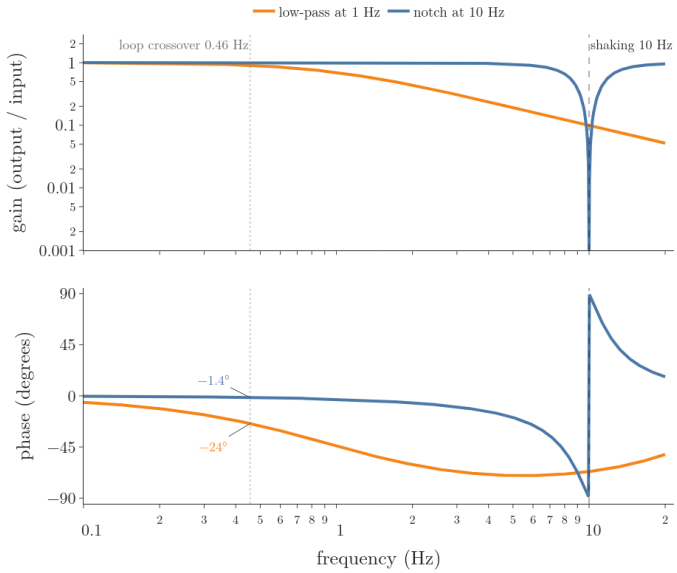


Figure 10: Two filters for the gyro, as frequency responses at 100 Hz. Top: gain (output over input, log axis) against frequency (Hz, log axis); bottom: phase in degrees. Orange: a first-order low-pass at 1 Hz. Blue: a notch at 10 Hz, 5 Hz wide. The dotted line marks the heading loop's crossover, 0.46 Hz; the dashed line the 10 Hz shaking. At the crossover the low-pass lags 24° and the notch 1.4°; at 10 Hz the low-pass passes 0.1 and the notch 0 (the log axis cannot show 0, so the blue curve is drawn down to its bottom edge, 0.001). Frequencies stop at 20 Hz

Figure 11 runs the three choices in time. Watch the red swing of 0.085 N · m, the orange one shrunk to 0.009 N · m and the blue one gone once the notch has settled.

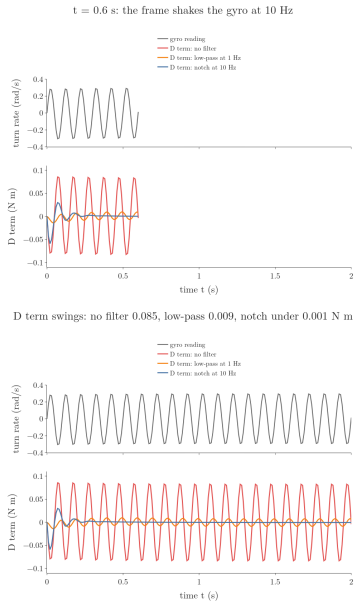


Figure 11: The notch filter in time. The derivative action uses the gyro, which carries the frame's 10 Hz shaking (top, grey, ±0.3 rad/s); the robot drives straight and the heading PID holds the heading at 0. Bottom: the derivative action with no filter (red, ±0.085 N m; the 0.2 N m limit lies above the chart), after a 1 Hz low-pass (orange, ±0.009 N m) and after a 10 Hz notch (blue, under 0.001 N m once the filter has settled, after about 0.2 s). Time runs frame by frame over 2 s

Gyro filter	Derivative action at 10 Hz	Lag at 0.46 Hz	Phase margin
none	±0.085 N · m	0°	75°
low-pass at 1 Hz	±0.009 N · m	24°	52°
notch at 10 Hz	under 0.001 N · m	1.4°	74°

The two filters answer the third problem of the Overview. The derivative filter cuts the encoder spikes from 0.043 to under 0.003 N · m and keeps 54° of phase margin; on the gyro, the notch removes the 10 Hz swing of 0.085 N · m and keeps 74°, where a low-pass would leave 52°.

In real stacks. PX4 cleans its gyro in a fixed order: a notch filter (parameters IMU_GYRO_NF0_FRQ and IMU_GYRO_NF0_BW, the centre and the width), then a low-pass (IMU_GYRO_CUTOFF), and the derivative action gets its own extra low-pass (IMU_DGYRO_CUTOFF), because "the D-term is most susceptible to noise" (PX4 controller diagrams; PX4 filter tuning). Its static notch is meant for narrow-band noise, "for example a bending mode of the airframe" (PX4 filter tuning). Autoware's speed controller low-passes its velocity error before the PID (`lpf_vel_error_gain: 0.9` in its parameter file). Each of these filters runs as a few lines of code at the controller's sample rate, like everything else in this Note.

7. Summary

Problem	Fix	Result on the robot
which gains	Ziegler–Nichols: raise k_p to k_c, read T_c, use the table	$k_c = 200$, $T_c = 0.40$ s; PI gains 90 and 270: fastest, but 0.032 s of delay margin
which gains	bump test, FOTD model, model-based rule	$K = 0.5$, $\tau = 0.094$ s, $T = 4.00$ s; gains 30.1 and 27.9: drop 0.039 m/s, back in 2.3 s, 0.27 s of delay margin
command step kicks	derivative on measurement ($\gamma = 0$)	no 2.85 N · m spike
overshoot after a step	set-point weighting, $\beta = 0.5$	overshoot 22.5 → 1.6 percent
encoder ticks	derivative filter, $T_f = T_d/N$, $N = 10$	spikes 0.043 → under 0.003 N · m, phase margin 54°
10 Hz frame shaking	notch filter at 10 Hz	swing 0.085 → under 0.001 N · m, phase margin 74°

- On the 1° ramp, the bump-test gains (30.1 and 27.9) cut the speed drop from 0.054 to 0.039 m/s and the recovery from 5.2 to 2.3 s, because they react harder while keeping a delay margin of 0.27 s; Ziegler–Nichols's 90 and 270 recover in 0.8 s but oscillate with 0.04 s more delay.
- The parallel and standard forms are one controller (the integral gain is k_p over T_i, the derivative gain k_p times T_d), so a tuner can work with two times that compare directly with the robot's time constants.
- The ideal speed model never oscillates, but the 20-sample average makes the measured speed 0.095 s late, so the real loop oscillates at $k_c = 200$ with period 0.40 s, where the lags add up to half a period.
- Ziegler–Nichols's table turns k_c and T_c into gains from one experiment, so it is quick, but it sees one point of the loop and gives fragile loops (FBS §11.3).
- A bump test gives K, τ and T, which match the drag, the mass and the averaging lag, so a rule built for this model can use all three and gives a third of the Ziegler–Nichols gain.
- The derivative of a command step is a spike (2.85 N · m, 14 times the limit), so the derivative acts on the measurement, which changes nothing between steps.
- The proportional action also jumps at a step, so weighting the command by $\beta = 0.5$ cuts the overshoot from 22.5 to 1.6 percent while the integral, on the full error, still reaches the command.
- The derivative action turns each 1.53 mrad encoder tick into a 0.043 N · m spike, so a low-pass whose time constant is the derivative time over N smooths it, at the price of phase margin (75° to 54° with $N = 10$).
- A notch removes one frequency, so it blocks the frame's 10 Hz shaking with a lag of 1.4° at the loop's 0.46 Hz, where a 1 Hz low-pass would lag 24° and leave a phase margin of 52°.

The robot now has tuned gains and a PID that neither kicks nor chatters. Everything in this Note ran on a computer that reads the sensors and updates the commands every 0.01 s: the average, the derivative as a difference over one sample, the filters as a few lines of code. What that sampling does to a controller designed as a continuous law, and how to choose the sample rate, is the subject of digital control.

8. Sources

Built from

- Åström, K. J. and Murray, R. M. (2021). *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed. Princeton University Press. §11.1 Basic Control Functions (eq. 11.1, integral and derivative times), §11.3 PID Tuning (Table 11.1, Fig. 11.7, eq. 11.9, eq. 11.10a), §11.5 Implementation (filtering the derivative, setpoint weighting, eq. 11.15, Fig. 11.13). Free: https://www.cds.caltech.edu/~murray/books/AM08/pdf/fbs-public_24Jul2020.pdf (FBS)
- Beauregard, B. (2011). "Improving the Beginner's PID – Derivative Kick", Project Blog, http://brettbeauregard.com/blog/2011/04/improving-the-beginner%e2%80%99s-pid-derivative-kick/ (Beauregard)
- PX4 Autopilot documentation, "Multicopter PID Tuning Guide (Manual/Advanced)", docs.px4.io, page config_mc / pid_tuning_guide_multicopter (PX4 tuning guide)
- PX4 Autopilot documentation, "MC Filter Tuning & Control Latency", docs.px4.io, page config_mc / filter_tuning (PX4 filter tuning)
- PX4 Autopilot documentation, "Controller Diagrams", docs.px4.io, page flight_stack / controller_diagrams (PX4 controller diagrams)
- Code and parameter files read on GitHub for the real-stack paragraphs (main or master branch, read 2026-10-12; slashes spaced so the paths can wrap):
  - PX4/PX4-Autopilot: src / lib / rate_control / rate_control.cpp;
  - ros-controls/control_toolbox: control_toolbox / src / pid.cpp, and control_toolbox / include / control_toolbox / pid.hpp;
  - ros-controls/ros2_controllers, folder diff_drive_controller / src: the files diff_drive_controller_parameter.yaml and odometry.cpp;
  - autowarefoundation/autoware_universe: control / autoware_pid_longitudinal_controller / config / autoware_pid_longitudinal_controller.param.yaml;
  - ApolloAuto/apollo: modules / control / controllers / lon_based_pid_controller / conf / controller_conf.pb.txt, and modules / control / control_component / controller_task_base / common / pid_controller.cc.

Other references

- MATLAB, "A PID Tuning Guide | Understanding PID Control, Part 4", YouTube, https://www.youtube.com/watch?v=sFOEsA0Irjs
- MATLAB, "Noise Filtering in PID Control | Understanding PID Control, Part 3", YouTube, https://www.youtube.com/watch?v=7dUVdrs1e18
- MATLAB, "Manual and Automatic PID Tuning Methods | Understanding PID Control, Part 6", YouTube, https://www.youtube.com/watch?v=qj8vTO1eIHo

- MATLAB, "A Better Way to Think About a Notch Filter | Control Systems in Practice, Part 5", https://www.mathworks.com/videos/control-systems-in-practice-part-5-a-better-way-to-think-about-a-notch-filter-1570179510885.html
- NPTEL (IIT Kharagpur), *Industrial Automation and Control*, Lecture 13 "Controller Tuning" and Lecture 14 "Implementation of PID Controllers", https://archive.nptel.ac.in/courses/108/105/108105063

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Ziegler–Nichols method (G-3093)	A PID tuning recipe from one experiment: with only proportional control, raise the gain until the loop oscillates steadily, read the critical gain and period, and turn them into gains with a fixed table (PI: $0.45k_c$ and $T_c/1.2$); quick, but it gives aggressive loops with small margins.
Model-based tuning (G-3097)	Choosing controller gains from a fitted model of the process, such as the FOTD model of a bump test fed into a rule like Feedback Systems eq. 11.10a; it uses more of the process's behaviour than Ziegler–Nichols and gives calmer, more robust gains.
Derivative kick (G-3098)	The spike in a PID's output when its command jumps: the error jumps within one sample, so its computed rate, and the derivative action, become huge (2.85 $\mathrm{N \cdot m}$ against a 0.2 $\mathrm{N \cdot m}$ limit for a 0.1 rad heading step); avoided by taking the derivative of the measurement.
Set-point weighting (G-3100)	Letting the proportional and derivative actions see only fractions β and γ of the command, while the integral keeps the full error; it softens the response to command steps (overshoot 22.5 to 1.6 percent with $\beta = 0.5$) without changing the response to disturbances.

Term	Meaning
Derivative filter (G-3101)	A first-order low-pass on the PID's derivative path with time constant $T_f = T_d/N$, N from 5 to 20; it smooths sensor noise such as encoder ticks before the derivative amplifies it, at the cost of some phase margin.
Notch filter (G-3102)	A filter that blocks a narrow band around one frequency and passes the rest, $(s^2 + \omega_0^2)/(s^2 + (\omega_0/Q)s + \omega_0^2)$; it removes a known vibration, such as a frame's 10 Hz bending seen by the gyro, with far less lag at the loop's own frequencies than a low-pass.
Ideal PID controller (G-3090)	The textbook PID law, $u = k_p e + k_i \int e\,dt + k_d\,de/dt$, with nothing added: no output limits, no filters, every action on the error; a real controller adds set-point weights, derivative on measurement, filters and anti-windup to it.
Parallel form (PID) (G-3091)	The PID controller written with one gain per action, k_p, k_i and k_d, whose three results add up, such as 0.2, 0.05 and 0.283 for the robot's heading loop (RO-060's gains); most robot software (ros2_control, Autoware, Apollo) takes its gains this way.
Standard form (PID) (G-3092)	The PID controller written as one gain times a bracket, $k_p(e + \int e\,dt/T_i + T_d\,de/dt)$, with the integral time T_i and derivative time T_d; changing k_p alone scales all three actions and keeps their timing.
Integral time T_i (G-3103)	The proportional gain over the integral gain, $T_i = k_p/k_i$, in seconds: the time a constant error takes to make the integral action as large as the proportional action (2 s for $k_p = 20$, $k_i = 10$), so a short one makes the integral take over quickly.
Derivative time T_d (G-3104)	The derivative gain over the proportional gain, $T_d = k_d/k_p$, in seconds: how far ahead the derivative action predicts the error along a straight line, since it applies now the change the proportional action will see T_d later.

Term	Meaning
Critical gain and critical period (G-3094)	The proportional gain k_c at which a loop oscillates without growing or dying, and the period T_c of that oscillation (200 N per m/s and 0.40 s for the robot's speed loop); they mark the point where the loop's lags add up to half a period, and the Ziegler–Nichols table uses them. Also called ultimate gain and period.
Bump test (G-3095)	An open-loop experiment: hold the process steady, step its input once (the drive force from 0 to 1 N) and record the output until it settles; its curve gives the gain, the apparent delay and the time constant used to tune a controller.
First-order-plus-time-delay (FOTD) model (G-3096)	A three-number model of a step response: nothing happens for a delay τ, then the output rises like a first-order system with gain K and time constant T (0.5 m/s per N, 0.094 s and 4.00 s for the robot); simple tuning rules are written for it. Also called first order plus dead time.
Derivative on measurement (G-3099)	Computing the D action from the rate of change of the measurement, $-k_d\, dy/dt$, instead of the error's; it equals the usual D action whenever the command is constant and ignores command jumps, so it removes the derivative kick.
PID controller (G-2965)	A controller whose command is the sum of a proportional, an integral and a derivative part of the error, K_p e + K_i (integral of e) + K_d de/dt; the commonest feedback controller, it reacts to the error now, its past and its trend; with K_d = 0 it is a PI controller, with K_i = 0 a PD controller.
Feedforward (G-3070)	A control force computed from the plan and the robot's model, such as mass times the planned acceleration plus drag ($ma_r + bv_r$), and sent before any error appears; it lets the robot follow a changing plan without first falling behind, while feedback removes what the model gets wrong.

Term	Meaning
Anti-windup (G-3078)	Any change to a controller with integral action that keeps the integral from growing while the actuator is saturated, so the command leaves the limit as soon as the error changes sign; clamping and back-calculation are the two common kinds.
Cascade control (G-3086)	Nesting feedback loops so that the outer loop's output is the inner loop's reference, such as a heading loop that commands the yaw rate for a gyro loop; it lets each loop be designed in turn, catches disturbances early and offers a place for limits.
Overshoot (step response) (G-2983)	How far a step response's peak goes past its final value, as a percentage of the final value, such as 52.7 % when a 0.5 rad turn peaks at 0.763 rad; it says how far a reply swings past its target.
Derivative (G-595)	The slope of a function at a point: how fast the output changes as the input changes there. It is zero at the lowest point of a smooth curve, which is how training finds a minimum.
Proportional gain (G-2959)	The factor K_p by which a proportional controller multiplies the error, such as 0.2 N m per rad or 20 N per m/s; a larger gain corrects harder, and for an undamped plant swings faster.
Integral gain (G-2964)	The factor K_i by which integral action multiplies the error's running sum, such as 0.05 N m per rad s; larger values remove a steady error sooner but swing more, and above a limit make the loop unstable.
Derivative gain (G-2961)	The factor K_d by which derivative action multiplies the error's rate of change, such as 0.283 N m per rad/s; too little leaves a swing, too much makes the response sluggish.
First-order system (G-2987)	A system whose output follows a differential equation with only a first derivative, such as the robot's speed under P control; it closes the same fraction of its remaining gap in every time constant and can never overshoot.

Term	Meaning
Time constant (G-2988)	The time a first-order system takes to close 63 % of the gap to its final value, m/(K_v + b) = 1 s for the robot's speed loop; it fixes the rise time (about 2.2 time constants) and the 2 % settling time (about 4).
Damping ratio (G-2992)	A second-order system's damping compared with the damping that just stops it from swinging, $\zeta = K_d/(2\sqrt{K_p I})$, 0.2 for D gain 0.08 and 1 for 0.4; it alone sets the shape of the reply and its overshoot.
Moving average (G-2267)	The list of averages a sliding window gives: each output is the mean of the numbers under the window, so sharp jumps are smoothed out.
Sample period and sample rate (G-3106)	The time T between two samples, and its inverse, the number of samples per second $f_s = 1/T$ in hertz, such as 0.01 s and 100 Hz; they set how often a digital controller sees the robot and acts.
Nyquist stability criterion (G-3041)	The test $Z = N + P$: the closed loop has Z poles in the right half-plane, where N is the number of clockwise turns of the Nyquist plot round -1 and P the number of the loop transfer function's poles there; stable when $Z = 0$, delays and open-loop-unstable plants included.
Critical point (G-3036)	The point -1 of the complex plane: wherever the loop transfer function equals -1 the closed loop has a pole, so the Nyquist criterion counts turns round it and the margins measure the distance to it.
Phase margin (G-3045)	How much extra phase lag the loop can take at its gain crossover frequency before the closed loop becomes unstable, 180ř + $\angle L(j\omega_{gc})$; typical designs keep 30° to 60°.
Delay margin (G-3046)	The smallest extra time delay that makes the closed loop unstable, the phase margin in radians divided by the gain crossover frequency (0.19 s for the robot's heading loop); it says how late sensor readings or commands may arrive.

Term	Meaning
Derivative action (G-2960)	A part of the command in proportion to the error's rate of change, estimated from two readings in a row; it pushes against fast changes, so it brakes a turn before the target and damps swings, but it magnifies sensor noise.
PD control (G-2962)	Feedback with a proportional part and a derivative part, such as K_p e + K_d de/dt; it acts like a spring and a damper, so it can bring a frictionless joint or heading to its target without swinging.
Integral action (G-2963)	A part of the command in proportion to the error's running sum (its integral over time); the sum grows while any error remains, so it removes the error a steady load leaves, at the cost of overshoot and, if too large, instability.
Pole (G-3000)	A value of s where a transfer function's denominator is zero, so the fraction blows up there, such as -2 (twice) for the PD heading loop; each pole adds a term e^(pt) to every reply, so its real part says fade or grow and its imaginary part how fast it swings.
Transfer function (control) (G-2999)	The number a linear system multiplies an exponential input e^(st) by, written as a function of s, such as G(s) = 1/(8s + 2) for the robot's speed and $10/s^2$ for its heading; it turns a differential equation into a fraction, so systems can be combined and read by algebra.
Gain crossover frequency ω_{gc} (G-3042)	The frequency at which the loop transfer function's gain is 1 (0 dB), such as 2.08 rad/s for the robot's heading loop; the phase margin and the delay margin are read there.
Frequency response (G-3020)	How a stable linear system answers a sine at each rate ω once its start-up swing has died away: the gain and the phase, equal to the length and angle of its transfer function at s = jω; it tells how much of a wave of each rate gets through and how late.

Term	Meaning
Bode plot (G-3023)	A pair of plots of a frequency response against the rate on a log scale: the gain in decibels and the phase in degrees; on it the pieces of a system add and the response is sketched with straight lines.